

TCGA Metadata

chu25

2025-09-02

Contents

```
suppressPackageStartupMessages({  
  library(SummarizedExperiment)  
  library(dplyr)  
})
```

```
# ===== CONFIG =====  
in_rds  <- "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment_filtered.rds"  
outdir  <- "/home/chu25/TCGA/"  
outfile <- file.path(outdir, "TCGA_clinical_metadata.csv")  
  
dir.create(outdir, showWarnings = FALSE, recursive = TRUE)  
message("[INFO] Reading SummarizedExperiment: ", in_rds)  
tcga_obj <- readRDS(in_rds)
```

```
# choose fields  
wanted <- c(  
  "tissue_or_organ_of_origin",  
  "primary_diagnosis",  
  "diagnosis_is_primary_disease",  
  "site_of_resection_or_biopsy",  
  "primary_site",  
  "disease_type",  
  "project_id",  
  "name"  
)  
  
# Support either a single SE or (rarely) a list of SEs; build one colData frame + sample_id  
if (inherits(tcga_obj, "SummarizedExperiment")) {  
  cd <- as.data.frame(colData(tcga_obj))  
  sample_id <- colnames(assay(tcga_obj))  
  # If project_id missing later, we won't have a per-object project fallback; that's fine.  
} else if (is.list(tcga_obj)) {  
  # Combine list of SEs into one metadata frame
```

```

parts <- lapply(names(tcga_obj), function(p) {
  se <- tcga_obj[[p]]
  stopifnot(inherits(se, "SummarizedExperiment"))
  cdf <- as.data.frame(colData(se))
  cdf$..source_project <- p
  cdf$..sample_id_tmp <- colnames(assay(se))
  cdf
})
cd <- do.call(rbind, parts)
sample_id <- cd$..sample_id_tmp
# Use ..source_project later if project_id is missing
} else {
  stop("[ERROR] Unsupported object: expected a SummarizedExperiment or a list of SEs.")
}

# Coerce non-atomic colData columns to simple vectors so CSVs are clean
coerce_simple <- function(x) {
  if (inherits(x, "Rle")) as.vector(x)
  else if (is.factor(x)) as.character(x)
  else if (is.list(x)) vapply(x, function(y) paste(y, collapse = ";"), character(1))
  else x
}
cd[] <- lapply(cd, coerce_simple)

# Helper to pick the first available column name from candidates
pick_col <- function(df, candidates) {
  nm <- candidates[candidates %in% colnames(df)]
  if (length(nm)) nm[1] else NULL
}

# Build export frame starting with sample_id (first column)
df <- data.frame(sample_id = sample_id, stringsAsFactors = FALSE)

# Fill requested fields directly if present; otherwise NA for now
for (nm in wanted) {
  df[[nm]] <- if (nm %in% names(cd)) cd[[nm]] else NA
}

# If we loaded a list of SEs and project_id is missing, use the source name
if (!"project_id" %in% names(cd) && "..source_project" %in% names(cd)) {
  if ("project_id" %in% names(df)) {
    df$project_id <- ifelse(is.na(df$project_id) | df$project_id == "",
                           cd$..source_project,
                           df$project_id)
  }
}

```

```

# Standardise project_id to "TCGA-XXXX" where possible
if ("project_id" %in% names(df)) {
  df$project_id <- as.character(df$project_id)
  df$project_id <- ifelse(
    grepl("^TCGA-", df$project_id), df$project_id,
    ifelse(grepl("^[A-Z]{4}$", df$project_id), paste0("TCGA-", df$project_id), df$project_id)
  )
}

# Fill "name" if missing, using fallbacks; finally fall back to sample_id
if ("name" %in% names(df)) {
  need_name <- is.na(df$name) | df$name == ""
  if (any(need_name)) {
    fallback_candidates <- c("barcode", "submitter_id", "sample_barcode", "Sample.ID", "sample", "b
    fb <- pick_col(cd, fallback_candidates)
    df$name[need_name] <- if (!is.null(fb)) cd[[fb]][need_name] else df$sample_id[need_name]
  }
}

# Report any requested fields that were not present in colData
missing_cols <- setdiff(wanted, colnames(cd))
if (length(missing_cols)) {
  message("[WARN] Missing in colData (filled with NA): ", paste(missing_cols, collapse = ", "))
}

# Write CSV
utils::write.csv(df, outfile, row.names = FALSE)
message("[OK] Wrote: ", outfile, " (", nrow(df), " rows, ", ncol(df), " columns)")

```